

Kunming AI+

An Applied Introduction to AI with PyTorch

Luca Ghafourpour



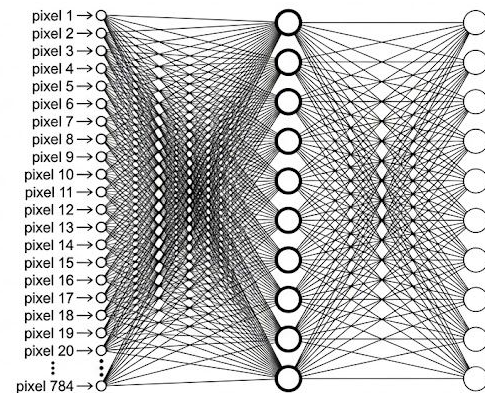
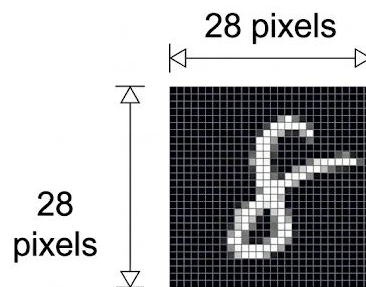
**UNIVERSITY OF
CAMBRIDGE**
Department of Engineering

Session Title	Topics
Workshop 1	VS Code, Conda and PyTorch
Practical Workshop 1	Constructing and training a neural network from scratch
Workshop 2	PyTorch modules for building and training neural networks
Practical Workshop 2	PyTorch modules for data loading
Workshop 3	Diagnosing training performance + <i>Weights and Biases</i>
Practical Workshop 3	Initialisation and regularisation
Workshop 4	Convolutional neural networks
Practical Workshop 4	Scikit Learn
Academic Lecture 1	Physics Informed Neural Networks
Academic Lecture 2	Neural Operators
Workshop 5	Complete end-to-end experiment
Practical Workshop 5	Open Q&A

Motivation behind convolutions

Problem 1: Too many parameters with FNNs

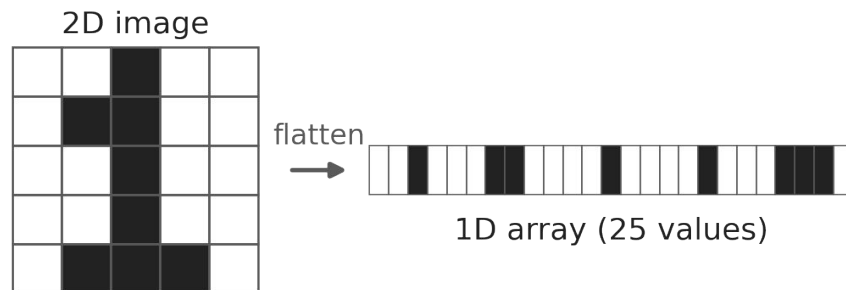
- A dense (fully-connected) layer connects every input pixel to every neuron.
- A small grayscale 28×28 image already needs 784 weights per neuron.
- A full HD RGB photo needs $1920 \times 1080 \times 3 \approx 6\text{M}$ weights per neuron!



! Fully-connected layers simply do not scale to images.

Problem 2: Spatial structure is lost

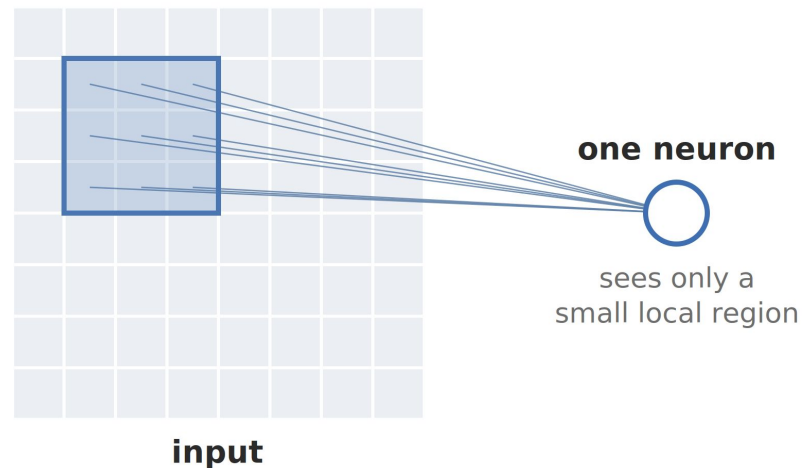
- To feed an image to a dense net, we flatten it into a 1D vector.
- After flattening, neighbouring pixels are no longer neighbours, and spatial correlations are lost.
- In reality, edges, corners and textures live in a 2D/3D layout.



We want a model that respects the geometry of the image.

Inspiration: Local receptive fields

- Hubel & Wiesel (1950s) showed that neurons in the visual cortex react to small local regions of the visual field.
- Each neuron has a small receptive field.
- Receptive fields of different neurons may overlap, together tiling the entire visual field.
- Simple cells detect edges, while later cells combine them into richer patterns.



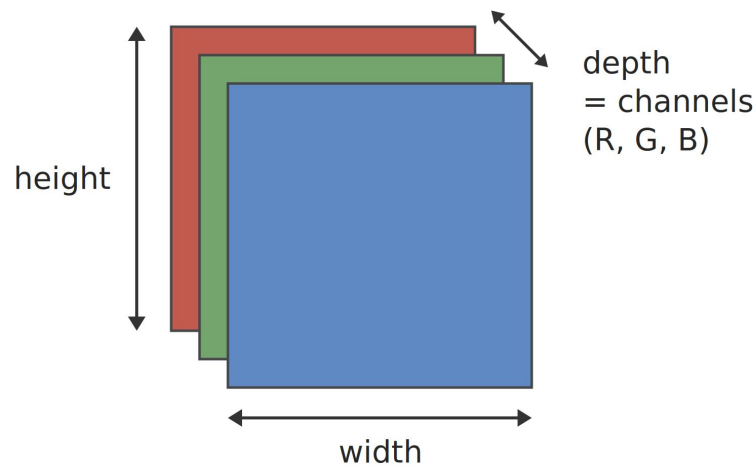
Connect each neuron to a small local patch, and reuse the same detector everywhere.

Convolution operator

Domain definition

- Input has dimension width $\times$ height $\times$ depth
- Depth = number of channels: RGB has 3, grayscale has 1.
- Channels can be anything stacked on a grid, e.g. colour, or velocity components of a flow field.

an image is a 3D volume



Convolution operation intuition

input kernel slide • multiply • sum feature map

0	0	3	0	0
0	2	2	2	0
1	3	3	3	1
0	3	0	3	0
0	1	3	1	0

 $\cdot$

1	0	-1
0	0	0
-1	0	1

 $=$

-1	0	1
-2	0	2
1	0	-1

input kernel feature map

0	0	3	0	0
0	2	2	2	0
1	3	3	3	1
0	3	0	3	0
0	1	3	1	0

 $\cdot$

1	0	-1
0	0	0
-1	0	1

 $=$

-1	0	1
-2	0	2
1	0	-1

Slide a small filter over the image.
At each position multiply with the patch underneath, sum, and add a bias.

Sweeping the whole image results in a **feature map**.

Convolution operation

$$(X * Y)_{i,j} = \sum_{u=0}^{F-1} \sum_{v=0}^{F-1} x_{i+u,j+v} \theta_{u,v} + b$$

X = the input, etc the input image or the previous layer's feature maps

θ = the filter weights, learned during training

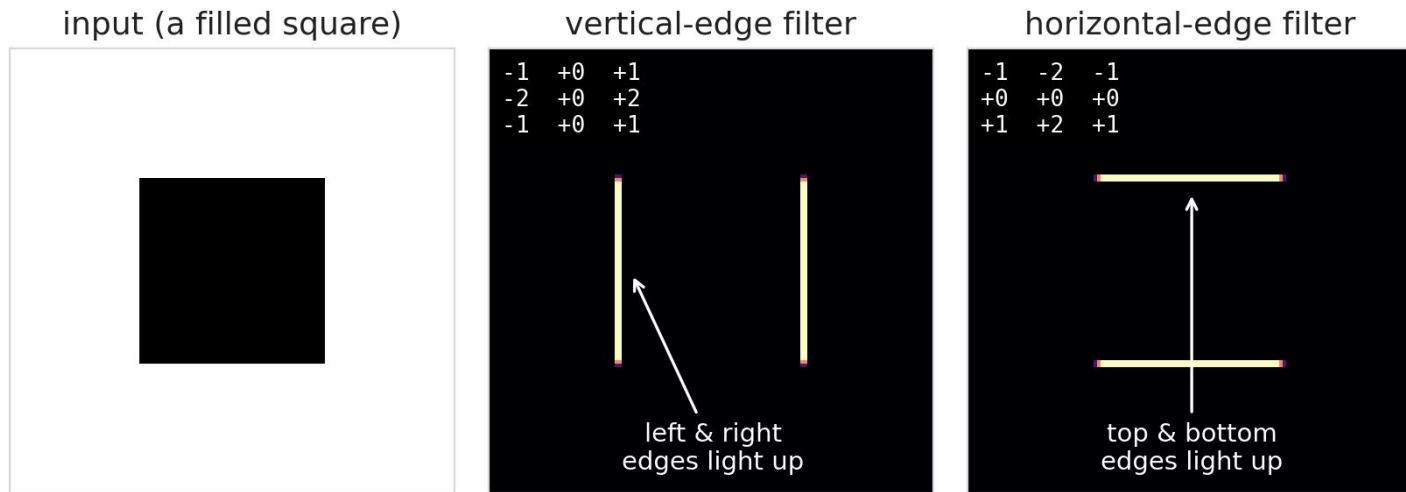
b = a bias added at every position

(i, j) = position in the output feature map

(u, v) = position inside the filter window

F = the filter size (F = 3 means a 3×3 filter)

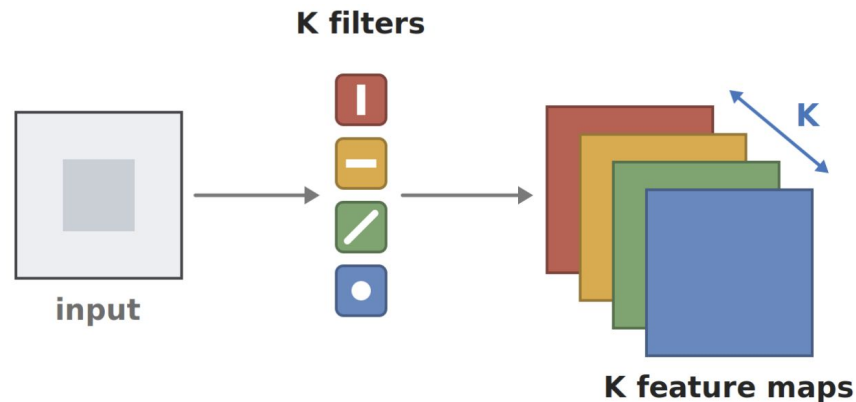
Filters as feature detectors



- Each filter is a small grid of weights
- Vertical-edge filters fire along the left/right sides, while horizontal-edge filters fire along the top/bottom
- **We do not design the filters. The network learns them by gradient descent. We only specify the dimension and number of filters per layer.**

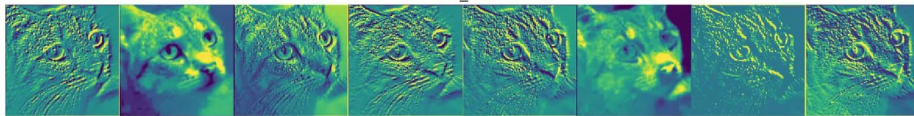
Stacking of filters

- One filter produces one feature map.
- Using K filters produces K feature maps.
- K determines the depth of a layer's output volume, and the input depth the next layer sees.
- All K filters look at the same input, in parallel.
- So more filters = more different detectors.
- Since all read the same raw input, all maps are at the same level of abstraction (e.g. all edge/colour/texture detectors).

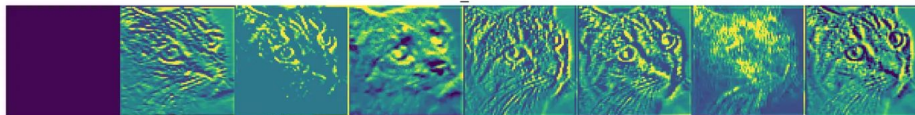


Depth adds abstraction

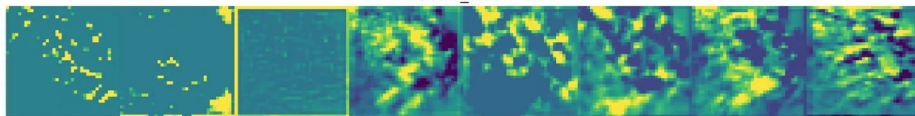
block1_conv1



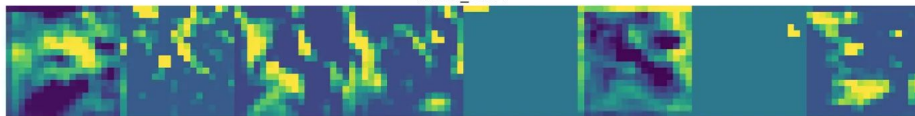
block2_conv1



block3_conv1



block4_conv1



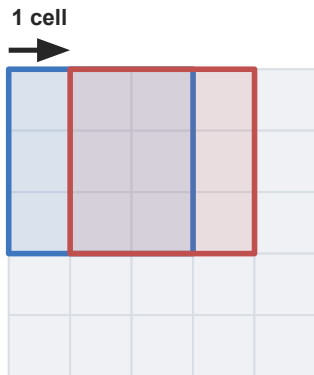
block5_conv1



- Stack many conv layers in sequence. Each layer's filters operate on the previous layer's feature maps.
- So features compose: edges → textures → parts → objects.
- Early layers respond to fine detail (edges, fur texture); deeper layers grow coarser, sparser, more abstract patterns.
- The receptive field grows with depth.

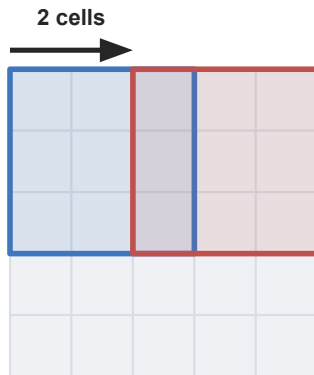
Stride and pooling

stride = 1



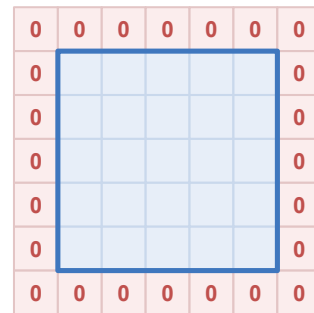
move 1 cell each step
more positions $\rightarrow$ larger output

stride = 2



move 2 cells each step
fewer positions $\rightarrow$ $\sim$ half the output

zero-padding



add a border of 0s
output keeps the input size

Stride = how far the filter jumps each step (stride 2 $\approx$ halves the output size).
Padding = a border (usually zeros) so the output keeps the input size.

Output map size in direction d

$$N_d = \frac{W_d - F_d + 2P_d}{S_d} + 1$$

- W = input size
- F = filter size
- P = padding
- S = stride

Worked example:

7×7 input, 3×3 filter, stride 1, no padding → $(7 - 3 + 0)/1 + 1 =$
5×5 feature map

Parameter sharing

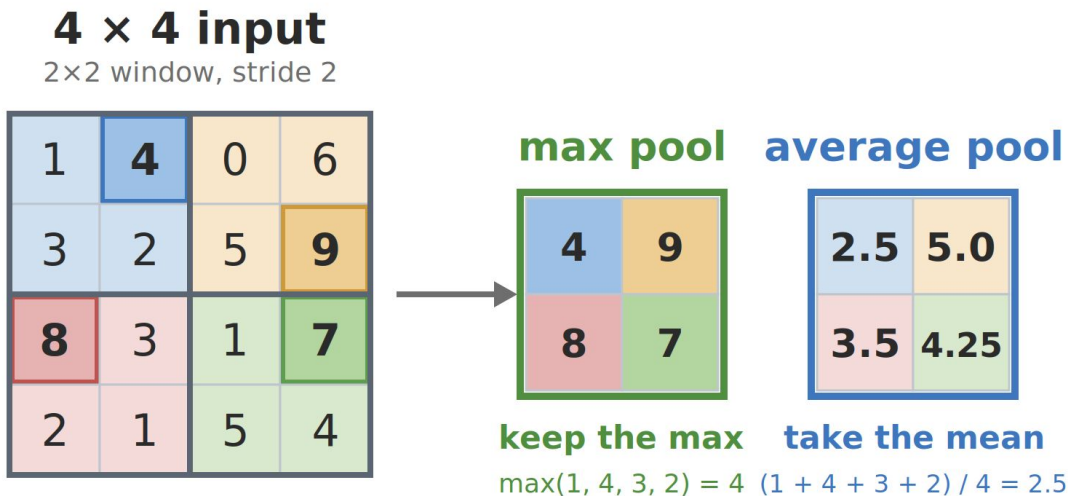
- Every neuron in a feature map shares the same filter weights.
- A convolutional layer's parameter count does not grow with image size.
- A feature that is useful in one part of an image, is useful everywhere, a property known as translation equivariance.

$$\text{Parameters} = \left(\prod_d F_d \times \text{number of channels} + 1 \right) K$$

Worked example:

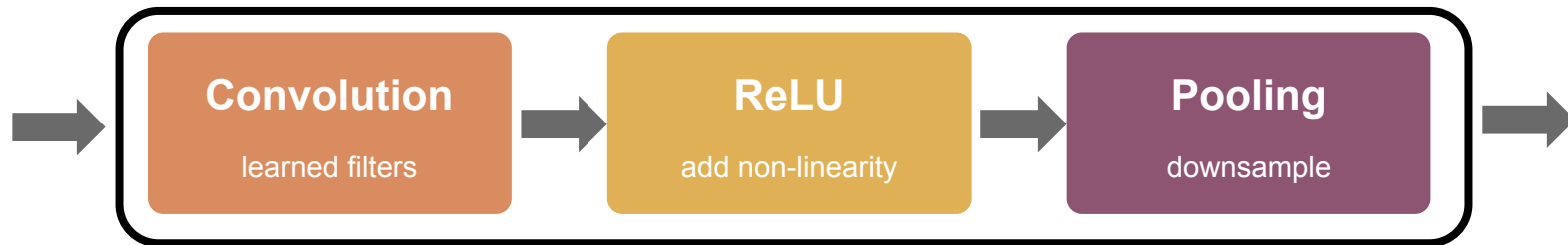
5×5 filter, 3 channels, 200 filters
 $(5 \cdot 5 \cdot 3 + 1) \times 200 = \mathbf{15,200 \text{ weights}}$

Pooling



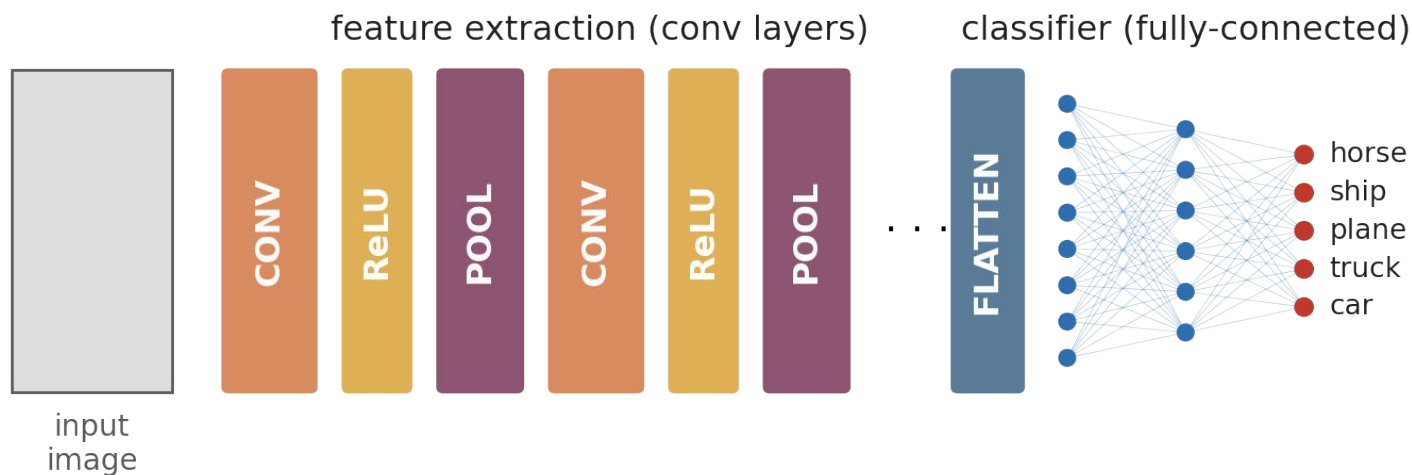
- Pooling downsamples each feature map → fewer parameters and more robustness to small shifts.
- Max pooling keeps the largest value in each region (the strongest response); average pooling takes the mean (a smoother summary).
- **A 2×2 window with stride 2 halves width and height; depth is unchanged — it acts on each channel independently.**

CNN block



- The convolution block learns the filters.
- Since convolution is a linear operation, we still need add a non-linearity.
- Pooling at the end to reduce dimensionality of deeper feature maps.
- Four hyperparameters: number of filters K , filter size F , stride S , padding P .
- Stack these blocks to build the feature extractor.

A complete CNN architecture



- Convolution and pool blocks extract features.
- Final feature map is flattened and a small MLP is used for classification.
- **The CNN and MLP is trained together end-to-end using backpropagation**

Rules of thumb

Input size

Divisible by 2 — 32, 64, 224 ...

Filters

3×3, stride 1, padded to keep size

Pooling

2×2 max-pool, stride 2

Width

Double the number of filters after each pool

Regularise

Data augmentation (shift, flip, rotate)

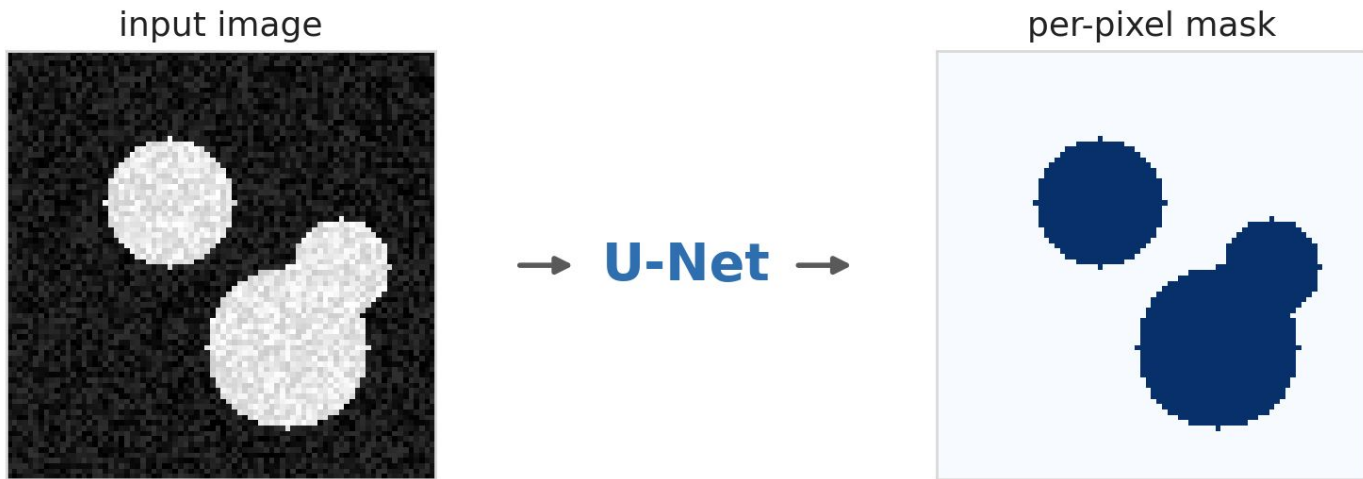
Regularise

Dropout ($\approx$ 40–50% in CNNs)

Time for some code...

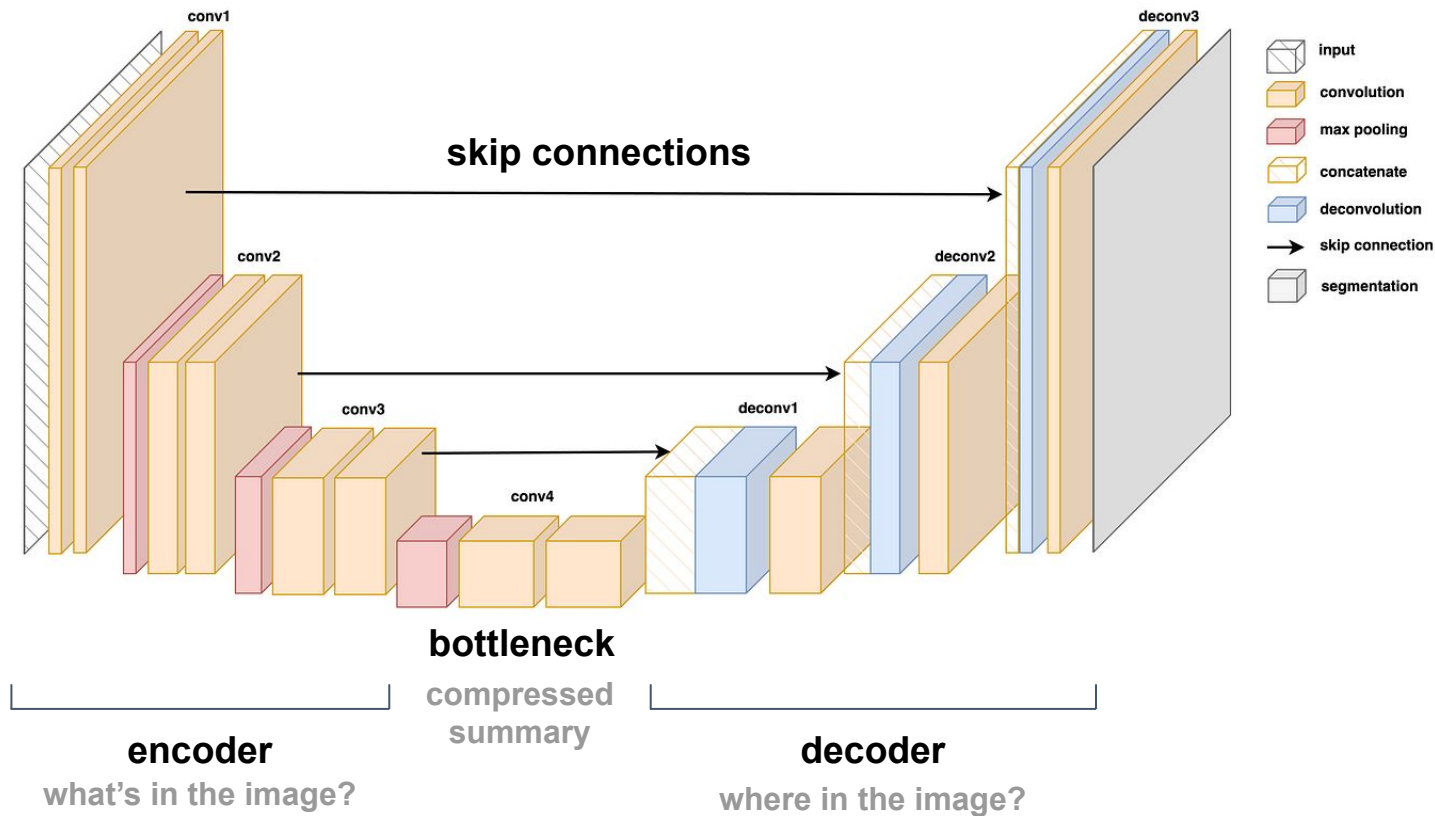
U-Nets

Pixel level generation



- A classifier gives one label per image; segmentation needs one label per pixel (e.g. which pixels are tumour).
- A U-Net outputs a map the same size as the input.

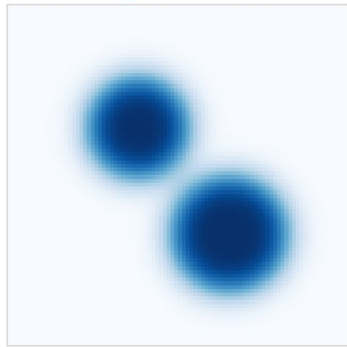
U-Net



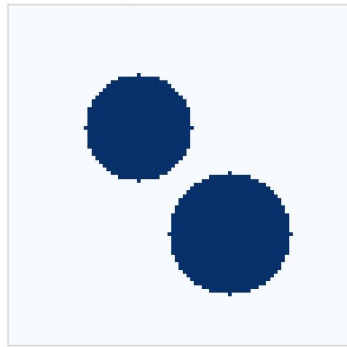
Why skip connections?

- Pooling throws away exact positions.
- By the bottleneck the network knows what is in the image, but not precisely where the boundaries are.
- Skip connections copy the encoder's early, high-resolution feature maps straight across to the decoder.
- Those early maps still hold sharp edges and fine detail, so the decoder can place boundaries exactly.
- **Result: crisp masks**

without skips:
blurry boundaries



with skips:
sharp boundaries



Recap

- 1 Convolution shares a small, learned filter across the image, resulting in few parameters.
- 2 A CNN stacks conv $\rightarrow$ ReLU $\rightarrow$ pool to turn pixels into features, then a dense head produces class scores.
- 3 By swapping the dense head for a decoder with skip connections (U-Net), you can predict a whole image, useful for tasks like segmentation.